

AI 应用能力文档

出海达人营销投前决策智能体

纪辉（Hugh） | 天津大学 计算机科学与技术 2023 级 | 应聘岗位：AI Builder（产品向）

把一句话投放 brief，变成一份可执行、可追责、带预算分配和风险证据链的达人投放清单，并用可复现的实验说清这份清单比「凭粉丝量选人」少浪费多少钱——以及这个结论有多稳。

这份文档里出现的每一个数字，都能在仓库里用一条命令复跑出来（固定种子、无网络依赖、两次运行产物逐字节相同）。凡是我无法复现的数字，我不会写进来。

在线 Demo：<https://185ab87138d0.aime-site.bytedance.net>
代码仓库与完整技术文档（docs/01~06，约 2.2 万字）随本文档一并提交

提交入口只有一个 [GitHub 仓库](#)：在线 Demo 链接、本文档 PDF、全部源码与实验产物、六篇技术分册（docs/01~docs/06，约 2.2 万字）都在仓库里，仓库 README 顶部有按阅读时长分流的导航。

0. 如果只看一页

做了什么：把一句话投放 brief（"3C 小家电新品要投北美，8 万美元，看转化"），变成一份可执行、可追责、带预算分配和风险证据链的达人投放清单，并用可复现的实验说清这份清单比行业最朴素的做法——[按粉丝量降序选人直到花完预算](#)——少浪费多少钱。

为什么是这个题：出海营销的钱，很大一部分不是浪费在投后优化不够，而是浪费在[投前选错人](#)。达人自称做 3C、实际内容在做美妆；粉丝 200 万但互动率 0.3%；受众 80% 在印尼而你要投沙特。这些问题投后看数据能发现，但钱已经花完了。

最硬的三个事实：

门禁能力稳定，不靠运气	水号识别 F1 0.7275 ± 0.0090 、AUC 0.8878 ± 0.0054 （12 个随机种子， $CV \leq 0.023$ ）
价值主张做了显著性检验	少浪费占预算 $21.5\% \pm 13.7\%$ （12 种子，95% CI 12.8%~30.2%），并承认 1/12 个种子跑输基线
Demo 不是录像，可以自己敲需求	打开就能写一条自己的投放 brief，召回、四层门禁、预算分配在浏览器里 当场重算 5,000 条 ；这套 TS 实现是 Python 版的独立第二实现，两者读同一份阈值表， 逐条比对 0 差异
假设不确定时有降级路径	预算模型的 decay 是建模假设。三档扫描下选中名单会变，于是名单按假设敏感性 分两层交付 ：核心层 162 人吃掉 90.2% 的预算可直接下单，剩下 9.8%（19 人）标记转人工

最诚实的三个弱项（我把它们放在第一页，不是附录）：

- 数据是我自己合成的**，不是真实达人数据。所有绝对数值只在这套数据生成假设内成立。这是这个作品最大的边界。
- 阈值敏感性判定为"不稳健"**：阈值 $\pm 20\%$ 时最大 F1 偏移 0.0582，我如实标了 `stable=False` 而不是调宽判据让它变 `True`。
- 我原本关于自己产品价值来源的判断，[被自己的实验推翻了](#)（详见 § 4.3）。
- 线上 Demo 没有后端服务**：门禁与预算是在浏览器里真算的，但 A1（brief 解析）线上走的是规则实现，A4（语义适配）读构建期固化产物——两处 LLM 能力在构建期真调过并留有 token 账，线上不调 endpoint（静态部署、不暴露 key、结果可复现）。我把"同一条 brief 交给 LLM 解析 vs 交给规则解析"的差异直接摆在页面上：30 个字段里 27 个一致，差的 3 个全在竞品回避——LLM 会按品类常识补出原文没写的竞品，规则版补不出来。

1. 为什么做这个题：与业务的对齐

我看蓝色光标的定位是「AI 驱动的全链路出海营销」，达人/网红营销是其中明确的一环，而 AI Builder（产品向）JD 的第一条职责就落在达人营销场景。所以我没有做一个通用的"AI 助手"，而是选了这条链路上**钱最容易白烧、且判断可以被结构化**的一个环节：**投前选人**。

选这个环节还有一个我认为更要紧的理由：**投放代理是低毛利生意**。在毛利率只有个位数的业务里，"帮你多赚 10%"很难证明，而"帮你少烧 10%"是直接落在利润上的。所以这个作品从一开始就不是往"效果提升"上做叙事，而是往**浪费控制与风险兜底**上做——这也决定了后面所有指标的选择。

2. 产品是什么

2.1 一条链路，六个 Agent

“一句话 brief ↓ A1 BriefAgent（LLM）自然语言 → 结构化 CampaignSpec ↓ A2 RecallAgent 按品类/地域/平台/粉丝层召回候选 ↓ A3 GateAgent（规则+分位阈值）四层质量门禁 G0/G1/G2/G3 → pass/review/reject + 证据链 ↓ A4 FitAgent 语义适配打分（人 × 单子） ↓ A5 BudgetAgent 带 5 条硬约束的预算分配 ↓ A6 AuditAgent 成本审计 + 反事实价值审计可执行清单：选谁、买几条、花多少、为什么、风险在哪”

2.2 四层门禁：把"这人能不能投"拆成可追责的判据

层	管什么	典型判据
G0	完整性	关键字段缺失/过期，数据不足就不给结论，而不是硬给一个
G1	真实性（水号）	互动率与同组分位背离、粉丝增长毛刺、评论质量、播放/粉丝比异常
G2	一致性	声称品类 vs 实际内容、受众地域/语言与投放目标错配
G3	品牌安全	敏感品类、争议历史、儿童向标准等

关键设计：**阈值是分组标定的，不是全局写死的**。instagram 头部号和 youtube 腰部号的"正常互动率"不是一个量级，全局阈值只会同时误杀一批、放过一批。所以阈值按 平台 × 粉丝层分组、用分位数标定（样本不足的组显式降级并上报）。

2.3 输出的是"可追责"，不是"一个分数"

每个被拒/被复核的达人都带证据链：命中哪条规则、该组的分位阈值是多少、他的实测值是多少。这一点是我认为产品化的关键——投放执行的人不会因为"模型说 0.31 分"就砍掉一个达人，但会因为"他的互动率在同组里排后 3%，而且评论 60% 是模板句"而砍。

3. 实测：决策质量

评测口径：5,000 个合成达人，ground truth 三档（pass 3058 / review 1034 / reject 908）。

反自证设计（这块是我花时间最多的地方）：ground truth 只允许评测代码读，门禁/预算/LLM

正式链路在物理上读不到。仓库里有 AST 静态扫描 +

泄漏哨兵测试守着这条线——因为我在这个项目上被"自证"坑过两次（§ 7）。

指标	结果
水号识别（严口径）	P=0.7714 / R=0.6750 / F1=0.7200 / AUC=0.8809
三分类	准确率 0.6590 / macro F1 0.6731
pass	P=0.9466 / R=0.5386
review	P=0.4052 / R=0.9613
reject	P=0.8104 / R=0.7203

pass 的精确率 0.9466、review 的召回 0.9613

是刻意的：这套门禁的产品定位是"宁可多送一批人去人工复核，也不要让水号直接过审花钱"。所以我接受 review 精确率只有 0.4052（会多送人工），换 pass 干净和 review 不漏。这不是指标没调好，是判据取向。

分欺诈类型（能力边界要说清楚）：买粉严召回 0.797 / AUC 0.919、互动农场 0.670 / 0.880、机器评论 0.640 / 0.893，而刷播放只有 0.473 / 0.794——播放量注入在我这套特征上最难抓，如实报出。

4. 实测：业务价值——卖的不是"多赚"，是"少烧"

4.1 招牌数字，和它诚实的版本

单次实验（定稿种子）：三个 campaign 合计 \$245,000 预算，少浪费 \$79,719（32.5%）。

这个数字很好看，但我对它做了显著性检验，结果是：它是 12 个种子里第 2 高的观测，落在均值 95% CI 之外。

	单种子	12 种子
少浪费占预算	-32.5%	-21.5% ± 13.7%（CI 12.8%~30.2%，1/12 跑输基线）

追加 24 种子复核进一步降到 19.1% ± 18.6%。所以我在所有对外材料里用的是 21.5% ± 13.7%，而不是继续挂 32.5%。

4.2 真正的价值不在均值，在方差

同一实验量化了另一件事：按粉丝量选人是一场高方差赌博。（口径：12 种子 × 3 campaign = 36 个观测）

	基线	KOXPilot
浪费率跨观测标准差	33.6%	4.7% (小 7.2 倍)
浪费率 > 50% 的观测	9/36 (25%) , 最惨一局 99.9%	0/36, 从未超过 14.8%

决定性证据在条件胜率上：把 36 个观测按"基线运气"分三档看——

基线浪费率	观测数	KOXPilot 胜率	平均少浪费
> 50% (基线踩大坑)	9	100% (9/9)	+\$57,480
5%~50% (中等)	12	91.7%	+\$13,296
< 5% (基线运气好)	15	仅 40%	-\$2,993

KOXPilot 输的 10 个观测里，基线浪费率最高只有 5.6%——也就是它只在"本来就没坑可踩"的局里输，而且输得很小；基线一旦踩坑，它 9/9 全胜。

所以这个产品的价值翻译成业务语言不是"帮你多赚 21.5%"，而是：把"四分之一概率烧掉一半预算"这个尾部风险，换成稳定的约 4% 浪费，代价是在本来就抽到好牌的局里小输一点。对低毛利的投放代理业务，消灭尾部灾难比提高均值更值钱。

两处必须自己先说的统计瑕疵，是叠加的、不是二选一：

>

① 有界比例。浪费率是 [0,1] 有界比例、KOXPilot 一侧还紧贴下界，尾部不正态会让 F 检验 p 值偏乐观。

>

② 伪重复 (pseudo-replication)。上面那 36 个观测是 12 种子 × 3 campaign，不是 36 次独立试验——同一种子下的三个 campaign 共享同一个达人库。伪重复会高估自由度、把 p 值进一步压小。所以合并后那个 $p=2.7e-21$ 我判为不可用，改报独立单位（种子）上的检验：三个 campaign 各自 $n=12$ ，方差比 p 分别为 $1.5e-07 / 1.3e-08 / 1.2e-06$ (F 值 51.6 / 81.8 / 34.9)。

>

结论因此以"标准差量级差异 + 极端频次计数 (9/36 vs 0/36) + 条件胜率"为主，这些都不依赖分布假设。两条限定与判据都写在 output/multiseed.json → B_variance_attribution.independence_unit 里，pooled_p_value_usable 字段直接标着 false，不是我临时补的说法。

4.3 这 21.5% 里，门禁到底占多少——一个推翻我自己判断的实验

原来的报告只有两条臂（基线 vs KOXPilot），于是"少浪费"这件事里混着两个原因：不把钱压在少数头部大号上（结构分散化）和识别水号并按质量排序（门禁）。谁贡献多少？答不上来。

所以我加了第三臂 diversified_no_gate：同一候选池、强制 KOXPilot 的分层/地域/单人配额，但排序只看"每美元名义曝光"，全程不读门禁判定与真实性折扣。于是 总差额 = (第三臂 - 基线) + (KOXPilot - 第三臂)。

12 种子 少浪费金额	mean ± std	为负的种子
合计	\$52.7k ± \$33.4k	1/12
其中 门禁与质量排序	\$40.0k ± \$12.6k (CV 0.31)	0/12
其中结构分散化	\$12.6k ± \$39.0k (CI 跨 0)	4/12

稳定的那一段是**门禁**，不是分散投放。而定稿单种子上看恰好相反（分散化 \$56.2k vs 门禁 \$23.5k）——我原本的猜测被自己的数据推翻了，这个结论我保留在仓库里而不是删掉重跑。

三条必须一起说的反面事实：

1. 第三臂的**绝对**有效曝光在 **12/12 个种子**上都高于 KOXPilot（它专挑 CPM 最便宜的长尾，KOXPilot 会主动放弃便宜但不对味的曝光）。所以三臂可比的口径是**浪费金额与有效曝光率**，绝对曝光数不是 KOXPilot 的优化目标。
2. 第三臂平均选 **516 人**（基线 18 / KOXPilot 181），真实采购里不可执行，它只是对照臂、不是方案。
3. "门禁贡献"其实是"门禁过滤 + 质量排序"的合计，两者共用同一批分数，**无法再拆细**——所以字段就叫 by_gating_and_quality_ranking，不谎报成纯门禁效果。

5. 该不该用大模型：用实验回答，不用立场回答

我不认为"用了大模型"本身是能力证明。所以我做了一组控制变量实验：**同一批 600 样本、同一 ground truth、同一 batch 切分**，唯一变量是 **system prompt**。

任务是最需要语义理解的那一层：**标签错配识别**（他声称做 3C，实际在做什么）。

Arm	Precision	Recall	F1	token
零成本规则对照组	0.4190	0.8571	0.5625	0
Prompt v1（角色+任务描述）	0.2367	0.8453	0.3627	434,018
Prompt v2（加约束与few-shot）	0.3554	0.7857	0.4889	467,327
Prompt v3（业务判据化）	0.6136	0.7976	0.6905	492,460
最优组合 v3 × GPT-5.5	0.6875	0.7857	0.7333	—

结论是反直觉的、也是我认为最有价值的一条：**v1 和 v2 都输给零成本规则**。花了 90 万 token，做得比一个 Jaccard 规则还差。v3 反超的原因不是"prompt 写长了"，而是把提示词从"你是一个专业的内容审核专家，请判断……"改成**业务判据**：明确写清"什么叫错配"（主品类漂移 vs 兼做副品类）、给出可判定的边界、要求先输出证据再输出结论。

这件事对产品的意义：LLM 在这条链路上的价值不取决于模型多强，取决于**你能不能把业务判据写清楚**。判据写不清，模型不如规则；判据写清了，模型比规则高 13pp。

5.1 成本账：混合架构不是省事，是算过的

方案	LLM 调用数	代价
全 LLM（每层每人都问模型）	888	贵、慢、不稳定
KOXPilot 混合（G0/G1 纯规则，仅 brief 解析与语义适配用 LLM）	225	调用量降 3.95×
全规则（不用 LLM）	0	标签错配 F1 从 0.6824 掉到 0.5625（-12pp）

真实 token 账（来自各 API 返回的 usage 字段，非估算）：172 次调用、110 万 token（prompt 53.7 万 / completion 56.7 万，其中 reasoning 40.3 万）。构建期真调模型，线上 Demo 读固化结果、零外部依赖。有 1 个 batch 调用失败，按"未覆盖"处理而不是填默认值——填默认值等于伪造模型输出。

6. 工程完备性

测试	745 条 pytest 全绿，含反自证元测试（AST 扫描确认生产代码读不到 gt）与文档数字防漂移测试
确定性	固定种子，make eval 两次运行 metrics.json 逐字节相同（产物刻意不含时间戳，任何指标变化都可归因到代码或数据）
前后端一致性	判定级 5,000/5,000、证据链级 255 条 reason、预算级 9 个臂（3 campaign × 3 臂，含第三臂对照）、审计级 3 个 campaign，diff 全为 0
假设可审计	预算模型里 decay=0.7 是建模假设不是观测值，补了三档敏感性扫描

关于 decay 扫描，结论我坚持两半一起说：价值结论不依赖这个假设（合计少浪费相对离差仅 1.4%、两段归因符号逐档不翻转），但具体选谁会变（选中名单金额加权重叠最低 0.651，低于我自设的 0.8 判据）。产物里专门有字段和测试堵住"我们验证了 0.7 是对的"这种过度解读——这条扫描回答不了哪一档更接近真实世界，那需要真实投放的重复触达数据。

但判据不达标之后，光承认"名单不唯一"是不够的——投手拿到的仍然是一份不知道哪里靠不住的清单，不确定性被承认了，成本却全转嫁给使用者。所以我给它接了一条降级路径：按"在几档 decay 上都被选中"把名单切成两层。

交付层	人数	参照档金额	占比	动作
核心层（三档全选中）	162	\$220,855	90.2%	可直接下单，入选不依赖 decay 取值
假设敏感层（仅部分档位选中）	19	\$23,962	9.8%	标记"重复触达折扣假设敏感"，转人工确认或改按单条采购，不自动执行

结论因此从"名单不唯一"变成一句可执行的话：[这个假设的不确定性只影响 9.8% 的预算，且已经定位到具体 19 个人](#)。产物见 `output/metrics.json` → `budget_decay_sensitivity.delivery_policy`（含逐 campaign 的人员与金额区间），两层必须无损划分正式名单、两层金额之和必须等于实付金额，由 `tests/test_decay_delivery_tiers.py` 钉住。

7. 我自己抓到并修掉的问题（13 条里挑 8 条）

一个只展示漂亮指标的作品是可疑的。仓库 README 里完整列了 13 条，这里挑最能说明问题的：

问题	表现	修完的代价
评测自证 ① ：标签错配的 ground truth 实际等价于"两个集合是否相交"	规则基线 $P=R=F1=1.000$	引入隐藏真实品类后，规则基线 F1 掉到 0.562
评测自证 ② ：观测模型整块替换，正例 Jaccard 只有 0 和 1	5 个 arm 召回 精确相同 (0.857)，尽管彼此预测差异 146/600	加逐品类观测噪声后，召回重新成为真实 tradeoff
消融表里有一句能被当场证伪的话	contributes 判定只看 <code>abs(delta)</code> 不看符号，于是 G1.6（关掉它 F1 反而涨 0.0149）被算成"有贡献"，产物还写着"所有 G1 规则都有边际贡献"	改三态判定（positive/negative/negligible）， 保留 G1.6 并如实标为负贡献
一个会爆炸的比率	有效曝光提升的分母是基线有效曝光，基线烧光时分母趋 0，实测出现 +3585× ，把跨种子均值抬到 +29,977%（中位数仅 +124.5%）	改用有界口径（有效曝光率百分点差），保留旧字段并标注不稳定原因
"我们用 LLM 做语义适配"这句话在正式链路里站不住	<code>llm_cache.json</code> 里有 780 条真实调用得到的 LLM 适配分，而 <code>make eval</code> 输出的每个数字都来自规则版	补量化审计：这 780 条打分落到三个 campaign 的去重候选池（222 人）里只覆盖 33 人（14.9%） 、与既有筛选 双算率 97.1% 、覆盖子集上 LLM 分只降不升 → 结论"继续用规则版" 由判据算出 （覆盖率 ≥90% 等三项全过会自动翻成可升格），并真跑了注入反事实（判定只翻 1 条、少浪费 \$735，不作为招牌数字）

这五条的共同点：[它们都不会报错，只会给我一个漂亮得不真实的结论](#)。我从中总结出的检查动作是——指标好得不合理时，第一反应应该是怀疑自证，而不是庆祝；多个方法结果完全相同时，先检查可分性是否已经退化。

7.1 收尾时又抓到另一类问题：结论正确，但没有产生动作

上面五条都是"数字不对"。收口阶段我发现还有一类更隐蔽的——[数字是对的，结论也是对的，但它到此为止了](#)。

[第一个](#)：敏感性扫描判出 `selection_stable = false`，我原来的处理是在文档里补一句"名单会随假设变动"。这句话没错，但它对使用者没有任何用——投手拿到的仍然是一份不知道哪里靠不住的清单，不确定性被我承认了，成本却全转嫁给了他。修法就是上面 § 6

那张分层交付表：把不确定性从"整份名单打个折扣"收敛成"具体 19 个人、9.8%"

的预算"。同一份数据，一个是免责声明，一个是分流规则。

第二个：decay_scan.py 里我特意把记录分成 blockers（会推翻结论的硬伤）和 caveats（不推翻结论但必须一起说的限定），源码注释还专门解释了"混成一个列表就会出现'扫过了没问题'这种听起来通过、实际掩盖限定条件的说法"——结果返回值里根本没有 caveats 这个键，算完就被丢掉了。本次实测 blockers 为空而 caveats 有 1 条，也就是说读者只会看到一片绿。讽刺的是这跟这个模块最初要修的那个 bug 是同一个毛病：常量注释替一段产物里不存在的输出背书。

第三个：README 里我写过一句"没有一个数字是手写进文档的"。这句话经不起追问——数字当然是手打进 markdown 的，只是来源于产物；真正的风险恰恰是产物重跑后数值变了，而文档里的字不会自己改。我没有把这句话改软，而是补了 tests/test_doc_numbers.py：把文档里的招牌数字逐条钉到产物字段上，连千分位、百分号、有效位都按人眼看到的字符串断言，还包括一条"文档声称的测试总数必须等于实际收集到的数量"。不改口气，改机制。

这三条我认为比前五条更值得说。前五条是技术功底问题，这三条是产品意识问题：一个结论如果不能推导出下一步动作，那它在产品里就还没做完。

8. 已知弱项与边界（不藏在附录）

弱项	数字	我的判断	
数据是合成的	—	这是最大边界。所有绝对值只在我的数据生成假设内成立。唯一能推翻或确认结论的实验是接一批真实达人 + 人工标注重跑	
头部大号识别差	follower_bucket=mega F1 0.467	系统性短板（11/12 种子进榜， $p=1.5e-07$ ），不是噪声	
单次弱项清单不可信	country=SA、`youtube\`	macro`	是采样噪声（ $p=0.132 / 0.803$ ）。典型 winner's curse：单次实验里最差的那几格，多数只是运气差
阈值敏感性不稳健	±20% 最大 F1 偏移 0.0582, stable=False	如实上报，没有调宽判据让它变 True	
刷播放最难抓	严召回 0.473	播放注入在现有特征上区分度不足	
A4 LLM 未接正式链路	覆盖率 14.9%	明确改口径为"规则版在生产链路、LLM 版构建期跑通并固化"，而不是让文档继续说 LLM	
名单不唯一	金额加权重叠最低 0.651	换 decay 假设就换一批人。已接降级路径：按假设敏感性分两层交付，不确定性收敛到 9.8% 的预算、19 个人（见 § 6）	

弱项	数字	我的判断	
预算分配非最优	—	是"边际性价比贪心 + 结构约束修正", 叠加 5 条约束后不保证最优, 我没做 MILP 对照, 所以说不出离最优差多少	

9. 我想说明的三件事

1. 我做产品的方式是"先能被追问，再能被展示"。这个作品里我花在评测、反自证、显著性检验上的时间，超过花在功能上的时间。因为一个投放决策产品，最终要被投放执行的人问"你凭什么让我砍掉这个达人"，被老板问"你说少烧的钱，怎么算的"。答不上来的产品不会被用。
2. 我把 AI 当成一个需要被度量的组件，不是一个卖点。这条链路上我真正拿 LLM 做的只有两件事（brief 结构化、语义适配），其余全是规则——因为我算过：全 LLM 是 3.95 倍调用量，而 G0/G1 用规则一分不差。同时我也用实验证明了：判据写不清的时候，LLM 是不如规则的。
3. 我愿意把不利的结论写进交付物。这份文档里有三处我自己的判断被自己的实验推翻（价值幅度被单次高估、价值来源不是分散化而是门禁、单次弱项清单大半是噪声），有一条负贡献规则我留在系统里如实标注。我认为这比多报几个百分点有用。

附录：复现方式

```
""bashgit clone <repo> && cd koxpilotpip install -e .
```

make all # 数据 → 门禁 → 预算 → 评测（固定种子，无网络依赖，约 1 分钟） make test # 745 条测试 make multiseed # 12 种子稳健性实验（纯确定性，约 30s）

需要 API key（构建期实验，可选）

```
make llm # 真实 LLM runner make promptbench # Prompt v1/v2/v3 × 双模型横评""
```

产物对照：output/metrics.json（六张表 + 反事实审计 + 诚实性清单）、output/multiseed.json（12 种子）、output/prompt_bench.json（Prompt 横评）、output/llm_bench.json（token 实账）。

文档索引：docs/01 架构 | docs/02 四层门禁 | docs/03 评测方法 | docs/04 Prompt 与成本 | docs/05 边界与已知问题 | docs/06 稳健性。